Spring AI RAG - Sample Knowledge Document

This is a selectable-text PDF created for testing document reading, chunking, metadata extraction, embedding generation, and OpenSearch indexing. It contains multiple sections and deliberately repeated technical vocabulary to support semantic retrieval experiments.

1. What is Retrieval-Augmented Generation?

Retrieval-Augmented Generation (RAG) combines information retrieval with language model generation. Instead of answering only from model parameters, a RAG application searches an external knowledge source and places relevant passages into the prompt. This can improve factual grounding and provide references to the original source.

Practical note: Retrieval-Augmented Generation (RAG) combines information retrieval with language model generation. Instead of answering only from model parameters, a RAG application searches an external knowledge source and places relevant passages into the prompt. This can improve factual grounding and provide references to the original source.

2. Document Ingestion

Document ingestion begins when a PDF is loaded. A PDF reader extracts text and may attach metadata such as page numbers and source names. The application should inspect extracted text before indexing because page headers, footers, and unusual line breaks can reduce chunk quality.

Practical note: Document ingestion begins when a PDF is loaded. A PDF reader extracts text and may attach metadata such as page numbers and source names. The application should inspect extracted text before indexing because page headers, footers, and unusual line breaks can reduce chunk quality.

3. Chunking Strategies

Chunking divides a large document into smaller passages. Fixed-size chunking uses a target length, while paragraph-aware chunking attempts to preserve topic boundaries. Token-based chunking controls the approximate number of tokens in each passage. Chunk overlap may preserve context across boundaries, but excessive overlap increases indexing cost.

Practical note: Chunking divides a large document into smaller passages. Fixed-size chunking uses a target length, while paragraph-aware chunking attempts to preserve topic boundaries. Token-based chunking controls the approximate number of tokens in each passage. Chunk overlap may preserve context across boundaries, but excessive overlap increases indexing cost.

4. Embeddings

An embedding model transforms text into a dense numeric vector. Texts with related meaning often have vectors that are close under a similarity measure. The same embedding model should normally be used for indexed chunks and user queries. Vector dimensions must match the index mapping.

Practical note: An embedding model transforms text into a dense numeric vector. Texts with related meaning often have vectors that are close under a similarity measure. The same embedding model should normally be used for indexed chunks and user queries. Vector dimensions must match the index mapping.

5. OpenSearch Indexing

OpenSearch can store chunk text, document identifiers, page numbers, and embedding vectors in an index. Vector search retrieves semantically similar passages. BM25 lexical search is useful for exact technical terms. Hybrid search can combine lexical and semantic retrieval to improve coverage.

Practical note: OpenSearch can store chunk text, document identifiers, page numbers, and embedding vectors in an index. Vector search retrieves semantically similar passages. BM25 lexical search is useful for exact technical terms. Hybrid search can combine lexical and semantic retrieval to improve coverage.

6. Spring AI Components

Spring AI provides document readers, document transformers such as TokenTextSplitter, embedding model abstractions, and vector store integrations. A typical ingestion pipeline reads a PDF into Document objects, splits them into smaller Documents, and calls VectorStore.add to embed and index the chunks.

Practical note: Spring AI provides document readers, document transformers such as TokenTextSplitter, embedding model abstractions, and vector store integrations. A typical ingestion pipeline reads a PDF into Document objects, splits them into smaller Documents, and calls VectorStore.add to embed and index the chunks.

7. Metadata and Provenance

Useful metadata includes document_id, file_name, page_number, chunk_index, ingestion_time, and content_hash. Some metadata comes from the reader, while application-specific fields must be added by the developer. Provenance makes it possible to show the source page alongside an answer.

Practical note: Useful metadata includes document_id, file_name, page_number, chunk_index, ingestion_time, and content_hash. Some metadata comes from the reader, while application-specific fields must be added by the developer. Provenance makes it possible to show the source page alongside an answer.

8. Retrieval and Answer Generation

When a user asks a question, the application embeds the query, retrieves relevant chunks, and constructs a prompt containing the evidence. A chat model then produces an answer. If the retrieved passages do not contain enough evidence, the application should acknowledge that limitation rather than inventing facts.

Practical note: When a user asks a question, the application embeds the query, retrieves relevant chunks, and constructs a prompt containing the evidence. A chat model then produces an answer. If the retrieved passages do not contain enough evidence, the application should acknowledge that limitation rather than inventing facts.

9. Evaluation

Retrieval quality can be measured using a small set of known questions and relevant passages. Recall at K checks whether relevant passages appear in the top K results. Developers should compare chunk size, overlap, and retrieval mode on the same evaluation set.

Practical note: Retrieval quality can be measured using a small set of known questions and relevant passages. Recall at K checks whether relevant passages appear in the top K results. Developers should compare chunk size, overlap, and retrieval mode on the same evaluation set.

10. Example Questions

What is RAG? Why do we chunk PDF text? What metadata should be stored with each chunk? How does OpenSearch combine vector and keyword search? What does VectorStore.add do in Spring AI? Why must embedding dimensions match the index mapping?

Practical note: What is RAG? Why do we chunk PDF text? What metadata should be stored with each chunk? How does OpenSearch combine vector and keyword search? What does VectorStore.add do in Spring AI? Why must embedding dimensions match the index mapping?